

Greedy Algorithms – Pattern Handbook (with code Templates)

Prepared by: Aman (Software Engineer II)

Last updated: Jan 22, 2026

1. Why Greedy Algorithms?

Greedy algorithms build a solution step-by-step by always taking the locally optimal choice with the hope that these choices lead to a globally optimal solution. They are especially effective when the problem exhibits the greedy-choice property (an optimal solution can be constructed from locally optimal choices) and optimal substructure (optimal solutions to subproblems compose into an optimal full solution).

This handbook organizes greedy problems into practical patterns you can quickly recognize during interviews or system/problem design. For each pattern, you'll find: when to use it, how to recognize it, correctness hints, and a compact Java template.

2. Proving Greedy Correctness (Quick Playbook)

Use one or more of these proof techniques:

- Greedy-choice property: Show that there exists an optimal solution that begins with the greedy choice. If an optimal solution starts differently, exchange the first step with the greedy step without hurting optimality.
- Exchange argument: Assume an optimal solution differs from the greedy one. Swap blocks/choices step-by-step to transform it into the greedy solution without decreasing quality.
- Stays-ahead argument: For every prefix (time, position, or step), show the greedy solution is at least as good as any other solution.
- Matroid framework: If the feasible solutions form a matroid, then sorting elements by weight and greedily adding feasible ones is optimal (e.g., Kruskal's MST).
- Counterexample check: Try to craft small counterexamples. If you can't after multiple attempts and the problem looks like a known pattern, greedy may be right.

3. Global Java Templates You Will Reuse

Comparator-based sorting:

```
// Sort by primary then secondary key
Collections.sort(list, (a, b) -> {
    if (a.key1 != b.key1) return Integer.compare(a.key1, b.key1);
    return Integer.compare(a.key2, b.key2);
});
```

```
// Arrays
Arrays.sort(arr, (x, y) -> Integer.compare(x[0], y[0]));
```

Min/Max Heap (PriorityQueue):

```
// Min-heap of integers
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// Max-heap of integers
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a,b) -> b - a);

// Custom object (min-heap by finish time)
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(it ->
it[1]));
```

Union-Find (Disjoint Set Union) for Kruskal:

```
class DSU {
    int[] p, r;
    DSU(int n){ p = new int[n]; r = new int[n]; for(int i=0;i<n;i++) p[i]=i; }
    int find(int x){ return p[x]==x? x : (p[x]=find(p[x])); }
    boolean union(int a, int b){
        a = find(a); b = find(b);
        if(a==b) return false;
        if(r[a] < r[b]) { int t=a; a=b; b=t; }
        p[b]=a; if(r[a]==r[b]) r[a]++;
        return true;
    }
}
```

4. Sort & Select: Interval Scheduling / Activity Selection

When to use:

Pick a maximum-size subset of non-overlapping intervals, or remove the fewest intervals to eliminate overlaps. Also appears in choosing compatible tasks/meetings.

How to identify:

Intervals [start, end]; objective is count/compatibility, NOT weighted profit. Sorting by earliest finish time and selecting greedily feels natural.

Correctness hint:

Earliest-finish-time stays ahead: it frees the timeline as soon as possible, leaving room for more intervals.

Java template:

```
int maxNonOverlapping(int[][] iv){
    Arrays.sort(iv, Comparator.comparingInt(a -> a[1]));
    int cnt=0, end=-1_000_000_000;
    for(int[] e: iv){
        if(e[0] >= end){ cnt++; end=e[1]; }
    }
    return cnt;
}

int eraseOverlap(int[][] iv){
    Arrays.sort(iv, Comparator.comparingInt(a -> a[1]));
    int keep=0, end=-1_000_000_000;
    for(int[] e: iv){ if(e[0] >= end){ keep++; end=e[1]; } }
    return iv.length - keep;
}
```

Example – Schedule the maximum number of non-overlapping meetings:

```
int[][] meetings = {{1,3},{2,4},{3,5},{0,2}};
System.out.println(maxNonOverlapping(meetings)); // 3
```

5. Interval Partitioning: Minimum Rooms/Platforms (Sweep Line + Min-Heap)

When to use:

Find minimum number of resources (meeting rooms, platforms) to handle all intervals without conflict.

How to identify:

Intervals with resource reuse after finish; overlapping intervals need separate resources. Sort by start, release finished via a min-heap on end time.

Correctness hint:

Heap tracks current overlaps; its maximum size equals the chromatic number of interval graph, which is optimal.

Java template:

```
int minRooms(int[][] iv){
    Arrays.sort(iv, Comparator.comparingInt(a -> a[0]));
    PriorityQueue<Integer> pq = new PriorityQueue<>(); // end times
    int ans=0;
    for(int[] e: iv){
        if(pq.isEmpty() || pq.peek() < e[0]) pq.add(e[1]);
        else pq.poll();
        ans = Math.max(ans, pq.size());
    }
    return ans;
}
```

```

    for(int[] e: iv){
        while(!pq.isEmpty() && pq.peek() <= e[0]) pq.poll();
        pq.offer(e[1]);
        ans = Math.max(ans, pq.size());
    }
    return ans;
}

```

Example – Minimum meeting rooms:

```

int[][] meetings = {{0,30},{5,10},{15,20}};
System.out.println(minRooms(meetings)); // 2

```

6. Sweep Line (Event Sorting)

When to use:

Track how many intervals are active over time, or find points where a property changes (e.g., skyline, arrows to burst balloons).

How to identify:

Create (+1) start and (-1) end events, sort by coordinate, sweep while maintaining a counter/heap.

Correctness hint:

Events process in order; at each boundary, the invariant about active intervals holds and yields optimal answers.

Java template:

```

int minArrows(int[][] balloons){
    Arrays.sort(balloons, Comparator.comparingInt(a -> a[1]));
    int arrows=0; long end = Long.MIN_VALUE;
    for(int[] b: balloons){
        if(b[0] > end){ arrows++; end = b[1]; }
    }
    return arrows;
}

```

Example – Min arrows to burst balloons:

```

int[][] balloons = {{10,16},{2,8},{1,6},{7,12}};
System.out.println(minArrows(balloons)); // 2

```

7. Fractional Knapsack (Sort by Value/Weight)

When to use:

Maximize value with capacity constraint when items can be taken fractionally (splittable).

How to identify:

Objective is linear; items divisible; sort by ratio value/weight and take greedily.

Correctness hint:

Linear programming relaxation is tight; exchange argument shows swapping in higher ratio never hurts.

Java template:

```
double fractionalKnapsack(int[] val, int[] wt, int W){
    int n = val.length;
    Integer[] idx = new Integer[n];
    for(int i=0;i<n;i++) idx[i]=i;
    Arrays.sort(idx, (i,j)-> Double.compare((double)val[j]/wt[j],
(double)val[i]/wt[i]));
    double res=0; int cap=W;
    for(int i: idx){
        if(cap==0) break;
        int take = Math.min(cap, wt[i]);
        res += (double)take * val[i] / wt[i];
        cap -= take;
    }
    return res;
}
```

Example – Fractional selection:

```
int[] val={60,100,120}; int[] wt={10,20,30};
System.out.println(fractionalKnapsack(val, wt, 50)); // 240.0
```

8. Job Sequencing with Deadlines (Max Profit, 1-Unit Jobs)

When to use:

Each job takes 1 unit time, has a deadline and profit. Maximize total profit executing before deadlines.

How to identify:

Sort jobs by profit descending; place each into the latest free slot before its deadline (scan or DSU).

Correctness hint:

Exchange argument: if a less-profitable job occupies a slot, swapping with a more-profitable job never decreases profit.

Java template:

```

int maxProfit(int[] d, int[] p){
    int n=d.length; Integer[] id=new Integer[n]; for(int i=0;i<n;i++) id[i]=i;
    Arrays.sort(id, (i,j)-> Integer.compare(p[j], p[i]));
    int maxD=0; for(int x: d) maxD=Math.max(maxD,x);
    boolean[] used = new boolean[maxD+1];
    int profit=0;
    for(int i: id){
        for(int t = Math.min(maxD, d[i]); t>=1; --t){
            if(!used[t]){ used[t]=true; profit+=p[i]; break; }
        }
    }
    return profit;
}

```

Example – Schedule profitable jobs:

```

int[] d={2,1,2,1}; int[] p={50,10,20,30};
System.out.println(maxProfit(d,p)); // 80

```

9. Optimal Merge Pattern: Connect Ropes / Files (Min Total Cost)

When to use:

Combine items pairwise where combining cost equals sum of sizes; minimize total cost (ropes, optimal merge).

How to identify:

Repeatedly combine the two smallest; maintain a min-heap.

Correctness hint:

Huffman-like argument: merging smallest first keeps large costs from repeating in future steps.

Java template:

```

int minCost(int[] a){
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for(int x: a) pq.offer(x);
    int cost=0;
    while(pq.size()>1){
        int x=pq.poll(), y=pq.poll();
        int c=x+y; cost+=c; pq.offer(c);
    }
    return cost;
}

```

Example – Connect ropes:

```

int[] ropes={2,4,3};
System.out.println(minCost(ropes)); // 14

```

10. Huffman Coding (Optimal Prefix Codes)

When to use:

Build minimal expected-length prefix code given symbol frequencies.

How to identify:

Use min-heap of frequencies; repeatedly merge two least frequent nodes.

Correctness hint:

Greedy with exchange argument on optimal prefix trees; identical to optimal merge pattern.

Java template:

```
static class Node { int f; Node l, r; Node(int f){this.f=f;} }
Node huffman(int[] f){
    PriorityQueue<Node> pq = new PriorityQueue<>(Comparator.comparingInt(n->n.f));
    for(int x: f) pq.offer(new Node(x));
    while(pq.size()>1){
        Node a=pq.poll(), b=pq.poll();
        Node c=new Node(a.f+b.f); c.l=a; c.r=b; pq.offer(c);
    }
    return pq.poll();
}
```

Example – Build code tree:

```
int[] freq={5,9,12,13,16,45};
Node root = huffman(freq); // tree built
```

11. Dijkstra's Shortest Path (Non-negative Edges)

When to use:

Shortest paths in graphs with non-negative weights.

How to identify:

Use min-heap; once a node is extracted (finalized), its distance is optimal.

Correctness hint:

Stays-ahead: the next node chosen has the smallest tentative distance, which cannot be improved via non-negative edges.

Java template:

```
int[] dijkstra(int n, List<int[]>[] g, int s){
    int[] dist = new int[n]; Arrays.fill(dist, Integer.MAX_VALUE/4); dist[s]=0;
    boolean[] vis = new boolean[n];
```

```

        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a->a[0]));
        pq.offer(new int[]{0,s});
        while(!pq.isEmpty()){
            int[] cur=pq.poll(); int d=cur[0], u=cur[1];
            if(vis[u]) continue; vis[u]=true;
            for(int[] e: g[u]){ int v=e[0], w=e[1];
                if(dist[v] > d + w){ dist[v] = d + w; pq.offer(new int[]{dist[v], v});
            }
        }
        return dist;
    }
}

```

Example – Run on adjacency list:

```

// g[u] holds {v, w}
List<int[]>[] g = new ArrayList[n]; for(int i=0;i<n;i++) g[i]=new ArrayList<>();
int[] dist = dijkstra(n, g, 0);

```

12. Minimum Spanning Tree (Kruskal & Prim)

When to use:

Connect all vertices with minimal total edge weight in an undirected connected graph.

How to identify:

Kruskal: sort edges by weight, union if they don't form a cycle. Prim: grow from a seed using min-heap.

Correctness hint:

Cut property & matroid: the lightest edge crossing any cut is safe.

Java template:

```

int kruskal(int n, int[][] edges){
    Arrays.sort(edges, Comparator.comparingInt(e->e[2]));
    DSU dsu = new DSU(n); int cost=0, cnt=0;
    for(int[] e: edges){ if(dsu.union(e[0], e[1])){ cost+=e[2]; if(++cnt==n-1)
break; } }
    return cost;
}

int prim(int n, List<int[]>[] g){
    boolean[] vis = new boolean[n];
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a->a[0]));
    pq.offer(new int[]{0,0}); int cost=0;
    while(!pq.isEmpty()){

```

```

        int[] cur=pq.poll(); int w=cur[0], u=cur[1];
        if(vis[u]) continue; vis[u]=true; cost+=w;
        for(int[] e: g[u]){ int v=e[0], wt=e[1]; if(!vis[v]) pq.offer(new
int[] {wt, v}); }
    }
    return cost;
}

```

Example – MST cost:

```

int[][] edges = {{0,1,1},{1,2,2},{0,2,3}};
System.out.println(kruskal(3, edges)); // 3

```

13. Gas Station (Single Pass Greedy)

When to use:

Find a start index to complete a circular route given gas[i] and cost[i], or detect impossibility.

How to identify:

If total gas < total cost, impossible. Otherwise, reset start when cumulative sum dips below zero.

Correctness hint:

Stays-ahead on prefix sums: any start before a negative prefix cannot be part of a feasible start.

Java template:

```

int canCompleteCircuit(int[] gas, int[] cost){
    int total=0, tank=0, start=0;
    for(int i=0;i<gas.length;i++){
        int delta = gas[i]-cost[i]; total+=delta; tank+=delta;
        if(tank<0){ tank=0; start=i+1; }
    }
    return total>=0? start : -1;
}

```

Example – Find start pump:

```

int[] gas={1,2,3,4,5}; int[] cost={3,4,5,1,2};
System.out.println(canCompleteCircuit(gas,cost)); // 3

```

14. Jump Game (Farthest Reach So Far)

When to use:

Decide reachability or minimum jumps (variant) given max jump lengths per index.

How to identify:

Maintain farthest index reachable while scanning left-to-right.

Correctness hint:

Stays-ahead: if current index is within farthest reach, updating farthest greedily never hurts.

Java template:

```
boolean canJump(int[] a){
    int far=0;
    for(int i=0;i<a.length && i<=far;i++){ far = Math.max(far, i + a[i]); }
    return far >= a.length-1;
}
```

Example – Reach end?:

```
int[] a={2,3,1,1,4};
System.out.println(canJump(a)); // true
```

15. Two-Pointer Pairing (Sort + Greedy Match)

When to use:

Pair items to minimize/limit resource usage (boats to save people, assign cookies, minimize difference).

How to identify:

Sort both sides; move pointers from ends to greedily match largest with smallest feasible.

Correctness hint:

Exchange argument: pairing the heaviest with the lightest feasible leaves maximum flexibility.

Java template:

```
int numRescueBoats(int[] p, int limit){
    Arrays.sort(p); int i=0, j=p.length-1, boats=0;
    while(i<=j){
        if(p[i]+p[j] <= limit) i++;
        j--; boats++;
    }
    return boats;
}

int findContentChildren(int[] g, int[] s){
    Arrays.sort(g); Arrays.sort(s); int i=0, j=0, cnt=0;
    while(i<g.length && j<s.length){
```

```

        if(s[j] >= g[i]){ cnt++; i++; j++; }
        else j++;
    }
    return cnt;
}

```

Example – Boats & Cookies:

```

int[] p={3,2,2,1}; System.out.println(numRescueBoats(p,3)); // 3
int[] g={1,2}, s={1,2,3}; System.out.println(findContentChildren(g,s)); // 2

```

16. Partition Labels (Extend to Last Occurrence)

When to use:

Split a string into max number of parts so that each letter appears in at most one part.

How to identify:

Record last index of each character, extend current partition to the farthest last occurrence seen.

Correctness hint:

Stays-ahead on positions: closing a partition only when the farthest last occurrence ends is safe.

Java template:

```

List<Integer> partitionLabels(String s){
    int[] last = new int[26]; for(int i=0;i<s.length();i++) last[s.charAt(i)-'a']
    = i;
    List<Integer> ans = new ArrayList<>(); int end=0, size=0;
    for(int i=0;i<s.length();i++){
        end = Math.max(end, last[s.charAt(i)-'a']); size++;
        if(i==end){ ans.add(size); size=0; }
    }
    return ans;
}

```

Example – Split string:

```

System.out.println(partitionLabels("ababcbacadefegdehijhklij")); // [9,7,8]

```

17. Greedy + Monotonic Stack: Remove K Digits (Lexicographically Smallest)

When to use:

Remove k digits to form the smallest possible number (or smallest subsequence).

How to identify:

While stack top > current digit and k>0, pop. Avoid leading zeros.

Correctness hint:

Local improvement strategy ensures each removal strictly decreases lexicographic order; stays-ahead.

Java template:

```
String removeKdigits(String num, int k){
    Deque<Character> st = new ArrayDeque<>();
    for(char c: num.toCharArray()){
        while(k>0 && !st.isEmpty() && st.peekLast()>c){ st.pollLast(); k--; }
        if(!st.isEmpty() && c=='0') st.offerLast(c);
    }
    while(k>0 && !st.isEmpty()){ st.pollLast(); k--; }
    if(st.isEmpty()) return "0";
    StringBuilder sb = new StringBuilder();
    for(char c: st) sb.append(c);
    return sb.toString();
}
```

Example – Smallest after removals:

```
System.out.println(removeKdigits("1432219", 3)); // 1219
```

18. Greedy + Stack: Smallest Subsequence of Distinct Characters

When to use:

Pick one occurrence of each character to form the smallest lexicographic sequence.

How to identify:

Track remaining counts; pop larger tops if they appear later again and are greater than current.

Correctness hint:

Exchange: replacing a larger earlier choice with a smaller current one keeps feasibility and improves lexicographic order.

Java template:

```
String smallestSubsequence(String s){
    int[] cnt = new int[256]; for(char c: s.toCharArray()) cnt[c]++;
    boolean[] used = new boolean[256];
    Deque<Character> st = new ArrayDeque<>();
    for(char c: s.toCharArray()){
```

```

        cnt[c]--; if(used[c]) continue;
        while(!st.isEmpty() && st.peekLast()>c && cnt[st.peekLast()]>0){
used[st.peekLast()]=false; st.pollLast(); }
        st.offerLast(c); used[c]=true;
    }
    StringBuilder sb=new StringBuilder(); for(char c: st) sb.append(c); return
sb.toString();
}

```

Example – Distinct smallest:

```
System.out.println(smallestSubsequence("cbacdcdbc")); // acdb
```

19. Deadlines + Durations: Max Tasks by Deadline (Course Schedule III)

When to use:

Each task has duration and deadline. Maximize the number of tasks finished before their deadlines.

How to identify:

Sort by deadline; keep a max-heap of durations. If total time exceeds current deadline, drop the longest task.

Correctness hint:

Exchange: replacing a longer earlier task with a shorter one can only help meet deadlines without reducing count.

Java template:

```

int scheduleCourse(int[][] c){
    Arrays.sort(c, Comparator.comparingInt(a->a[1]));
    PriorityQueue<Integer> maxH = new PriorityQueue<>((a,b)->b-a);
    int time=0;
    for(int[] x: c){
        time += x[0]; maxH.offer(x[0]);
        if(time > x[1]) time -= maxH.poll();
    }
    return maxH.size();
}

```

Example – Schedule max courses:

```

int[][] c={{100,200},{200,1300},{1000,1250},{2000,3200}};
System.out.println(scheduleCourse(c)); // 3

```

20. Coin Change (Canonical Systems Only)

When to use:

Make change using the fewest coins; greedy works for canonical denominations (like US/India: 1,2,5,10,20,50...).

How to identify:

Repeatedly take the largest coin $\leq$ remaining amount. Beware: for arbitrary sets (e.g., {1,3,4}), greedy may fail.

Correctness hint:

For canonical systems, exchange arguments hold; otherwise use DP (unbounded knapsack).

Java template:

```
int greedyCoins(int[] coins, int amount){
    Arrays.sort(coins); int cnt=0;
    for(int i=coins.length-1;i>=0;i--){
        if(coins[i]==0) continue;
        cnt += amount / coins[i];
        amount %= coins[i];
    }
    return amount==0? cnt : -1;
}
```

Example – Canonical change:

```
int[] coins={1,2,5,10};
System.out.println(greedyCoins(coins, 28)); // 6 (10+10+5+1+1+1)
```

21. Greedy for Set Cover (ln n-Approximation)

When to use:

Choose minimum subsets to cover all elements; NP-hard, use greedy approximation.

How to identify:

Repeatedly pick the set that covers the largest number of uncovered elements.

Correctness hint:

Harmonic bound $(1+\ln n)$ -approximation via potential function; optimality not guaranteed.

Java template:

```
List<Integer> setCover(List<Set<Integer>> sets, Set<Integer> universe){
    Set<Integer> covered = new HashSet<>();
    List<Integer> pick = new ArrayList<>();
    while(covered.size() < universe.size()){
        int best=-1, gain=-1;
        for(int i=0;i<sets.size();i++){
```

```

        int g=0; for(int x: sets.get(i)) if(!covered.contains(x)) g++;
        if(g>gain){ gain=g; best=i; }
    }
    if(best==-1) break;
    pick.add(best);
    for(int x: sets.get(best)) covered.add(x);
}
return pick;
}

```

Example – Cover universe:

```
// Returns indices of chosen sets (approximate)
```

22. Sort + Greedy Adjustments: Make Array Elements Unique (Min Increments)

When to use:

Make all elements unique by incrementing elements; minimize total increments.

How to identify:

Sort and push each element to at least prev+1.

Correctness hint:

Stays-ahead invariant: after processing i-th element, resulting sequence is strictly increasing with minimal moves.

Java template:

```

int minIncrementForUnique(int[] a){
    Arrays.sort(a); int moves=0, need=Integer.MIN_VALUE;
    for(int x: a){
        if(need==Integer.MIN_VALUE) { need=x; }
        need = Math.max(need, x);
        moves += Math.max(0, need - x);
        need++;
    }
    return moves;
}

```

Example – Unique array:

```

int[] a={3,2,1,2,1,7};
System.out.println(minIncrementForUnique(a)); // 6

```

23. Merge Intervals (Sort by Start)

When to use:

Combine overlapping intervals into minimal number of disjoint intervals.

How to identify:

Sort by start and greedily extend the running end to the max end seen so far.

Correctness hint:

Stays-ahead: merged end is always the farthest possible given processed intervals.

Java template:

```
int[][] merge(int[][] iv){
    Arrays.sort(iv, Comparator.comparingInt(a->a[0]));
    List<int[]> res = new ArrayList<>();
    int s=iv[0][0], e=iv[0][1];
    for(int i=1;i<iv.length;i++){
        if(iv[i][0] <= e) e = Math.max(e, iv[i][1]);
        else { res.add(new int[]{s,e}); s=iv[i][0]; e=iv[i][1]; }
    }
    res.add(new int[]{s,e});
    return res.toArray(new int[0][]);
}
```

Example – Merge:

```
int[][] iv={{1,3},{2,6},{8,10},{15,18}};
int[][] m = merge(iv); // [[1,6],[8,10],[15,18]]
```

24. Pitfalls: When Greedy Fails (Use DP/Backtracking)

Common traps:

- 0/1 Knapsack – requires DP; greedy by ratio fails (classic counterexample).
- Weighted Interval Scheduling – need DP; greedy by finish time is not optimal with weights.
- Coin Change with non-canonical denominations – greedy may produce suboptimal number of coins.
- Traveling Salesman Problem – nearest neighbor (greedy) not optimal in general.

25. Recognition Checklist (Interview Speed-Guide)

- Are decisions irrevocable and naturally ordered in time or by a key? Consider sort + sweep/selection.
- Does taking the smallest/largest element repeatedly feel right? Consider heaps (min or max).

- Do feasible solutions grow by adding elements without violating constraints? Matroid-like → greedy often works.
- Are edge weights non-negative? Dijkstra is applicable; negative edges break greedy optimality.
- Intervals on a line? Think earliest finish, partitioning with heaps, or sweep line.
- Deadlines involved? Sort by deadline; keep a heap to drop the worst offender when over budget.
- String selection for lexicographic minimality? Greedy + stack with last-occurrence tracking.
- Pairing problems with capacity limits? Sort + two pointers pairing heavy with light.

26. Closing Tips & Practice

- 1) Always try to craft a 4–6 element counterexample against your greedy idea. If none surface, you may be on the right track.
- 2) Write the invariant explicitly (what is always true after processing i items?). If you can maintain it locally, your proof is close.
- 3) Prefer $O(n \log n)$ solutions via sorting/heap; many greedy problems reduce to sorting once.
- 4) For graphs: non-negative → Dijkstra; spanning → MST (Kruskal/Prim). With negative edges, consider Bellman–Ford or SPFA (not greedy).